TRABALHO DE PÓS-GRADUAÇÃO

Cybersecurity Offensive: Pentest & Red Team

Lab 2: Broken Object Level Authorization (BOLA)

Identificação e exploração de BOLA em endpoint de API REST

Aluno: Victor Caldas dos Santos

1. Objetivo

Este trabalho tem como objetivo identificar e demonstrar uma vulnerabilidade de Broken Object Level Authorization (BOLA) em uma API REST de laboratório (vuln_api), avaliando como a aplicação trata autenticação e autorização de forma independente, e evidenciando o risco de um usuário autenticado acessar objetos pertencentes a outros usuários.

2. Metodologia

- Ambiente: API vuln_api executada localmente (http://127.0.0.1:5000).
- Ferramenta: curl (Parte B) e Burp Suite Community Edition, aba Repeater (Parte C), a partir de um terminal Kali Linux.
- Credencial de teste: token Bearer de usuário comum (alice-lab-token), correspondente à usuária Alice (id 1, role user).
- Etapas: (1) verificação do comportamento da API sem autenticação; (2) verificação do reconhecimento da identidade autenticada; (3) requisição ao próprio objeto do usuário autenticado; (4) requisições ao mesmo endpoint alterando apenas o identificador do objeto no path, mantendo o mesmo token.

Parte B: Entendendo a identidade da API

Requisição sem autenticação

```
curl -i http://127.0.0.1:5000/api/me
```

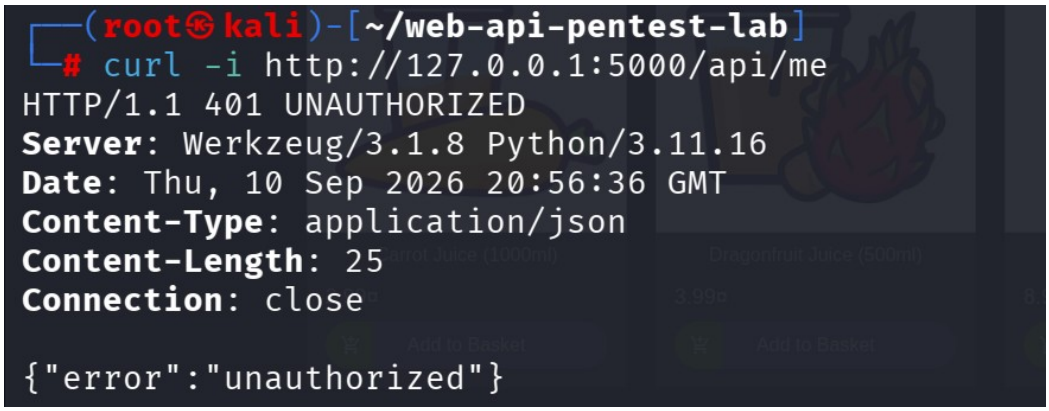


Figura 1. Requisição sem token de autenticação.

Status retornado: HTTP 401 Unauthorized, com corpo {"error":"unauthorized"}. A API rejeita corretamente requisições sem credencial.

Requisição com token de Alice

```
curl -i \
  -H "Authorization: Bearer alice-lab-token" \
  http://127.0.0.1:5000/api/me
```

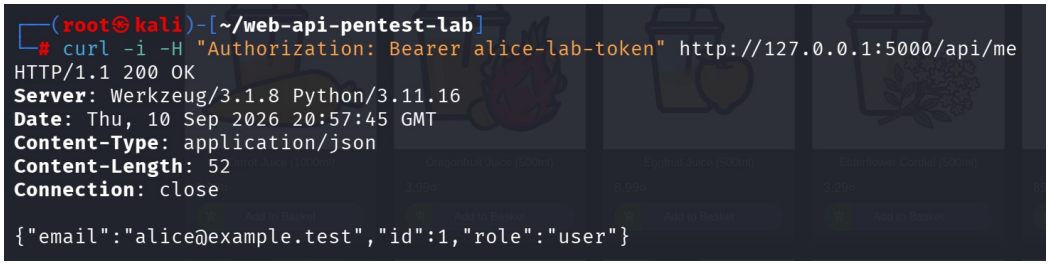


Figura 2. Requisição autenticada com o token da Alice.

Status retornado: HTTP 200 OK, com corpo {"email":"alice@example.test","id":1,"role":"user"}. A API reconhece corretamente Alice como usuária autenticada, com id 1 e papel user.

Parte C: Identificando BOLA

Requisição legítima: objeto do próprio usuário

```
GET /api/users/1 HTTP/1.1
Host: 127.0.0.1:5000
Authorization: Bearer alice-lab-token
```

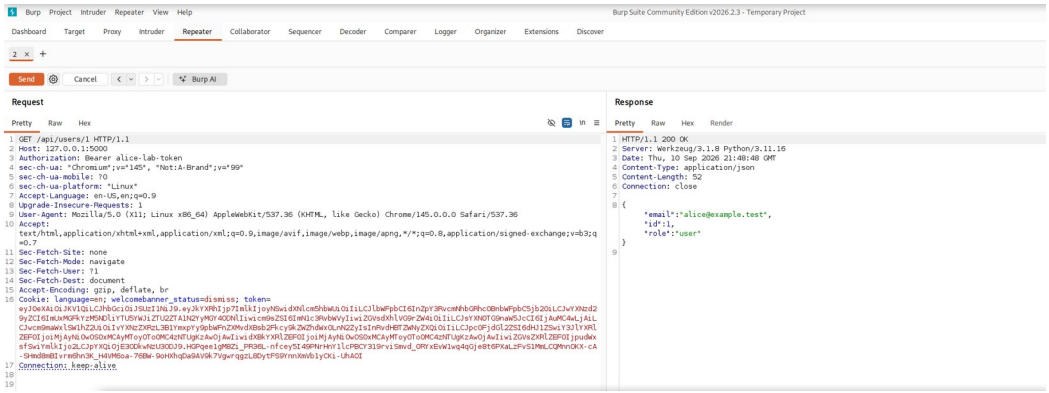


Figura 3. Alice acessando o próprio objeto (users/1), via Burp Suite Repeater.

Status: HTTP 200 OK, com corpo {"email":"alice@example.test","id":1,"role":"user"}, dado coerente com a identidade autenticada.

Exercício: alterando apenas o identificador do objeto

Mantendo exatamente o mesmo token de Alice, o identificador no path foi alterado para outros valores, sem qualquer outra modificação na requisição.

```
GET /api/users/2 HTTP/1.1
Host: 127.0.0.1:5000
Authorization: Bearer alice-lab-token
```

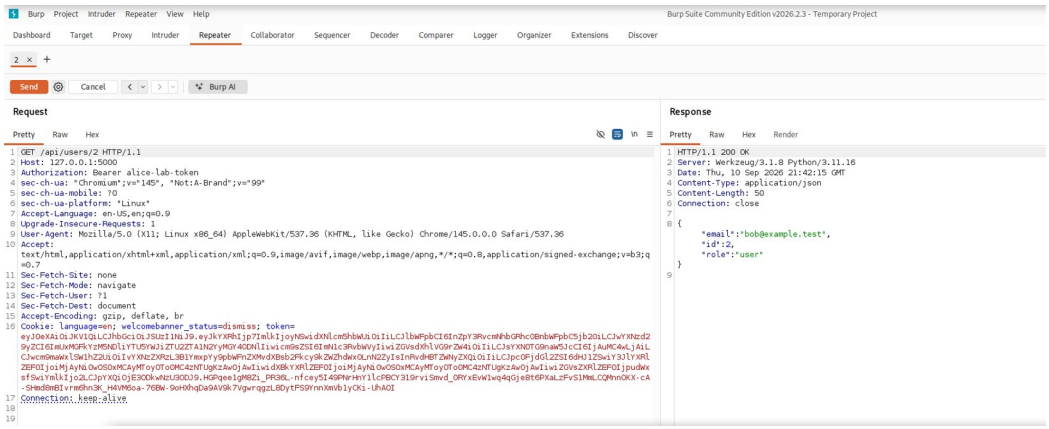


Figura 4. Mesmo token de Alice acessando users/2 (objeto de Bob), via Burp Suite Repeater.

Status: HTTP 200 OK, com corpo {"email":"bob@example.test","id":2,"role":"user"}. A API retornou dados de outro usuário (Bob) para uma requisição autenticada como Alice.

```
GET /api/users/3 HTTP/1.1
Host: 127.0.0.1:5000
Authorization: Bearer alice-lab-token
```

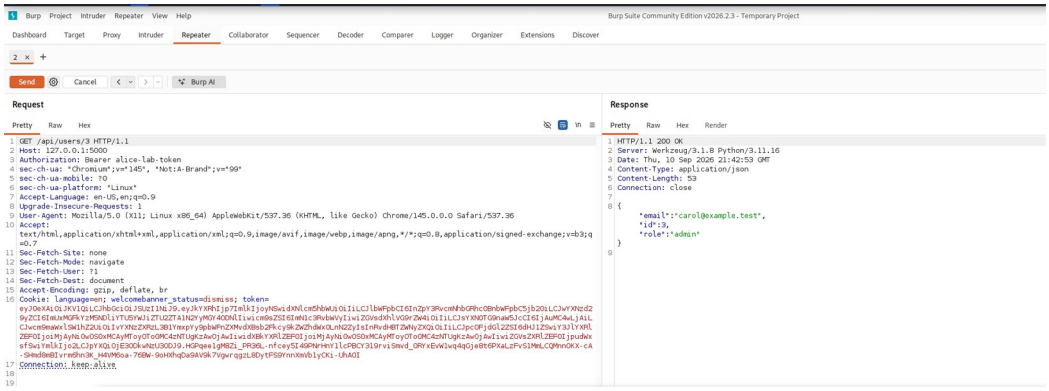


Figura 5. Mesmo token de Alice acessando users/3 (objeto de Carol, administradora), via Burp Suite Repeater.

Status: HTTP 200 OK, com corpo {"email":"carol@example.test","id":3,"role":"admin"}. Com o mesmo token de um usuário comum, foi possível obter os dados de uma conta administrativa.

Tabela resumo dos testes

Objeto acessado	Token usado	Status	Dado retornado
users/1 (próprio)	alice-lab-token	**200**	alice@example.test, id 1, role user
users/2 (Bob)	alice-lab-token	**200**	bob@example.test, id 2, role user
users/3 (Carol)	alice-lab-token	**200**	carol@example.test, id 3, role admin

Respostas às perguntas da Parte C

1. Qual elemento da requisição identifica o objeto acessado? O parâmetro {id} no path da URL (/api/users/{id}).

2. Esse valor é controlável pelo cliente? Sim. O identificador é enviado literalmente na URL pelo cliente, sem qualquer derivação a partir do token de autenticação.

3. O servidor deveria confiar apenas nesse valor? Não. O servidor autentica corretamente o token, mas não verifica se o objeto solicitado pertence ou está autorizado para o sujeito autenticado, tratando validade do token e autorização de acesso ao objeto como a mesma coisa.

Critério de sucesso

O critério de sucesso do exercício foi atingido: com o mesmo token de um usuário comum (Alice), foi possível acessar os objetos de outros usuários (Bob e Carol) sem que o servidor realizasse qualquer verificação de autorização correspondente, incluindo o acesso indevido a uma conta com papel administrativo.

Parte D: Evidência técnica

Caso 1: acesso ao objeto de Bob

Ator autenticado: Alice (id 1, role user).

Objeto autorizado: usuário 1 (Alice).

Objeto acessado indevidamente: usuário 2 (Bob).

Pré-condição: token válido de usuário comum (alice-lab-token).

Resultado: HTTP 200 OK com os dados de Bob (bob@example.test, id 2, role user), usuário diferente do autenticado.

Caso 2: acesso ao objeto de Carol (administradora)

Ator autenticado: Alice (id 1, role user).

Objeto autorizado: usuário 1 (Alice).

Objeto acessado indevidamente: usuário 3 (Carol).

Pré-condição: token válido de usuário comum (alice-lab-token).

Resultado: HTTP 200 OK com os dados de Carol (carol@example.test, id 3, role admin), incluindo a exposição de uma conta com privilégios administrativos a partir de um token de usuário comum.

Conclusão da evidência

O problema identificado é de autorização quebrada, e não apenas de autenticação quebrada, porque a etapa de autenticação funcionou corretamente em todos os testes: o token da Alice foi validado e sua identidade foi corretamente reconhecida pelo servidor em todas as chamadas. O que falhou foi a etapa seguinte, na qual o servidor deveria ter verificado se a identidade autenticada tinha permissão sobre o objeto específico solicitado antes de retornar os dados. Como essa verificação não existe, qualquer usuário autenticado, independentemente de quem seja, consegue acessar qualquer objeto apenas alterando o identificador na URL, caracterizando Broken Object Level Authorization (BOLA).

Parte E: Impacto e risco de negócio

No teste realizado, o dado exposto foi identidade e papel do usuário (e-mail e role), mas o mesmo endpoint, sem correção, exporia qualquer dado que o modelo de usuário armazene. O problema escala diretamente com IDs previsíveis e sequenciais: como /api/users/{id} usa inteiros incrementais, um atacante pode automatizar a enumeração de toda a base de usuários (id 1, 2, 3, ..., N) sem precisar adivinhar nada, exfiltrando a base inteira em poucos minutos. Se o endpoint retornasse endereço, CPF ou dados bancários, o impacto passaria de exposição de identidade para exposição de dados pessoais sensíveis e financeiros, viabilizando fraude, engenharia social direcionada e roubo de identidade. Se a operação fosse PUT ou DELETE em vez de GET, o BOLA deixaria de ser apenas vazamento de leitura e passaria a permitir modificação ou exclusão não autorizada de contas de terceiros, incluindo administradores, o que representa uma escalada de gravidade, de confidencialidade para integridade e disponibilidade. Em um sistema real, isso representaria risco regulatório (violação da LGPD/GDPR por exposição de dados pessoais sem controle de acesso adequado, sujeita a multa) e risco reputacional (perda de confiança de clientes e parceiros, além de possível dever de notificação de incidente).

Parte F: Proposta de remediação

O servidor não deveria decidir o acesso apenas com base na validade do token; ele precisa de uma checagem de autorização em nível de objeto a cada requisição, considerando os seguintes elementos:

- Identidade do sujeito: extrair do token, já validado na etapa de autenticação, qual é o usuário autenticado (no caso, id 1, Alice), e não confiar em nenhum identificador vindo do cliente para essa finalidade.
- Objeto solicitado: identificar claramente qual recurso está sendo pedido, no caso o usuário correspondente ao id informado no path (/api/users/{id}).
- Relacionamento entre sujeito e objeto: antes de retornar qualquer dado, verificar se o sujeito autenticado é o dono do objeto solicitado (sujeito.id igual ao objeto.id) ou possui alguma relação que justifique o acesso.
- Papel e permissões: caso a aplicação preveja acesso ampliado, como um administrador consultando qualquer usuário, essa exceção deve ser explícita e verificada no servidor a partir do role do sujeito autenticado, nunca assumida.
- Política de autorização no servidor: a regra final deveria ser equivalente a: permitir GET /api/users/{id} somente se id for igual ao id do sujeito autenticado, ou se o role do sujeito autenticado for admin. Essa lógica deve ser aplicada sempre no backend, nunca no cliente, e idealmente centralizada em uma camada de autorização reutilizável, para que toda rota que receba um identificador de objeto no path passe pela mesma verificação.

8. Classificação OWASP

A vulnerabilidade identificada é classificada como OWASP API1:2023, Broken Object Level Authorization, primeira posição do OWASP API Security Top 10, caracterizada por endpoints que aceitam identificadores de objeto controlados pelo cliente sem validar se o sujeito autenticado tem autorização sobre o objeto específico solicitado.

9. Conclusão

O teste demonstrou que a API vuln_api autentica corretamente os usuários, mas não realiza nenhuma verificação de autorização em nível de objeto no endpoint GET /api/users/{id}, permitindo que um usuário comum, com um único token válido, acesse dados de qualquer outro usuário apenas alterando o identificador na URL, incluindo contas administrativas. Esse tipo de falha é especialmente perigoso por ser simples de explorar, difícil de perceber em testes superficiais e capaz de expor a base de usuários inteira caso os identificadores sejam previsíveis. A correção passa necessariamente por implementar, no servidor, uma checagem explícita de propriedade ou permissão entre o sujeito autenticado e o objeto solicitado, em vez de confiar apenas na validade do token.